

GPU Computing Final Project

Ettore Saggiorato

247178

Universita' di Trento

Trento, Italy

ettore.saggiorato@studenti.unitn.it

Abstract— This report evaluates three methods for transposing square, non-symmetric, dense matrices on CUDA-capable devices using NVIDIA's Cooperative Groups, compared against the cuBLAS library. Experiments, conducted on the University of Trento GPU Cluster, reveal that Cooperative Groups provide no significant performance benefits for this task. While cuBLAS is not always the fastest, it demonstrates greater stability in throughput, highlighting its robustness for dense matrix transposition.

I. INTRODUCTION

The aim of this project is to develop an efficient algorithm for transposing square, dense matrices on a GPU, utilizing NVIDIA's Cooperative Groups[1]. The matrices considered in this work are dense, non-symmetric, square matrices with dimensions $\mathcal{N} \times \mathcal{N}$, where $\mathcal{N}$ is a power of 2, ranging from 2^2 to 2^{15} .

Dense matrices are central to numerous applications in fields such as computational mathematics, numerical analysis, machine learning, data science, physics, engineering, and finance. Matrix transposition, a fundamental operation in linear algebra, is widely used across these domains for tasks such as data manipulation and optimization.

As matrix transposition is a core operation, optimizing its performance is critical for enhancing computational efficiency. This report investigates various strategies for leveraging NVIDIA's Cooperative Groups to optimize matrix transposition on the GPU, and evaluates their effectiveness in improving performance.

A. Instructions for Reproducibility

The code and instructions for reproducing the results discussed in this report are available on GitHub.

II. STATE OF THE ART

The state-of-the-art solution for transposing dense matrices on NVIDIA GPUs is provided by cuBLAS ([2]). This library supports dense matrices and vectors and offers GPU-accelerated, optimized routines for fundamental linear algebra oper-

ations. Matrix transposition is implemented through two key functions, tailored for single and double precision operations:

- **cublasSgeam**: The primary function for single precision matrix transposition. It allows users to specify matrix operations, including transposition, by utilizing the CUBLAS_OP_T operation flag.
- **cublasDgeam**: The corresponding function for double precision matrix transposition, analogous to cublasSgeam.

This project focuses on single precision operations.

A. Cooperative Groups

Cooperative Groups is a programming model introduced to provide more flexible and structured ways of managing and synchronizing threads. It allows to organize threads into hierarchical groups, enabling fine-grained control over parallelism and synchronization at different levels, like within a warp, a block, or across the entire grid. Cooperative Groups enable thread organization by creating logical grouping of threads beyond the default block and grid structure, flexible synchronization by synchronizing subsets of threads within a group instead of all threads in a block and scalability by supporting scalable algorithms by enabling cooperation between threads at multiple levels. Given the foundational nature of this work, the primary focus will be on experimenting with different algorithms and benchmarking their performance rather than introducing groundbreaking advancements in the field.

III. CONTRIBUTION AND METHODOLOGY

Given the capabilities of Cooperative Groups (CG), which provide APIs for granular thread management, and the relatively straightforward requirements of a dense square matrix transpose (as opposed to more complex operations like full reductions), it is anticipated that kernels developed with Cooperative Groups for this task may face performance limitations.

This project explores two primary approaches:

- **Intra-block synchronization using Cooperative Groups**: This method is expected to perform comparably to traditional implementations using `__syncthreads`, as

the synchronization scope is confined to threads within the same block.

- **Inter-block synchronization using Cooperative Groups:** This method is expected to underperform due to the significant overhead introduced by grid-wide synchronization, which is inherently more complex and time-consuming.

To evaluate these approaches, four kernels were developed:

- One reference kernel for copying operations.
- Three kernels for matrix transposition.

A. Synchronization Strategies and Kernel Descriptions

1. Kernels with intra-block synchronization

Three kernels, described in Algorithm 1, Algorithm 2, and Algorithm 3, utilize the `cg::sync(cta)` function from the Cooperative Groups (CG) API to synchronize threads within a block. Here, `cta` represents the cooperative thread block group, which encompasses all threads within the block. Functionally, `cg::sync(cta)` operates similarly to the traditional `__syncthreads()` mechanism.

2. Kernel with inter-block synchronization

The kernel described in Algorithm 4 demonstrates the challenges associated with inter-block synchronization for matrix transposition. This kernel employs `cudaLaunchCooperativeKernel` to synchronize across the entire grid. Due to the inherent constraints of inter-block synchronization, this kernel is limited to matrices with dimensions $\mathcal{N} \times \mathcal{N}$ with $\mathcal{N} \in [2^2, 2^5]$. The narrow range is a limitation of the current implementation due to the absence of subtitling.

```

1 set shared tile[tile_dim][tile_dim]
2 let cta ← cg::this_thread_block()
3 let x ← blockIdx.x * tile_dim + threadIdx.x
4 let y ← blockIdx.y * tile_dim + threadIdx.y
5 let width ← gridDim.x * tile_dim1
6 for i ← 0 to tile_dim by block_rows do:
7 | tile[tIdx.y + i][tIdx.x] ← idata[(y + i) * width + x]
8 cs::sync(cta)
9 for i ← 0 to tile_dim by block_rows do:
10 | odata[(y + i) * width + x] = tile[tIdx.y + i][tIdx.x];

```

Algorithm 1: CUDA Copy Coop

```

1 set shared tile[tile_dim][tile_dim]
2 let cta ← cg::this_thread_block()
3 for i ← 0 to tile_dim by block_rows do:
4 | tile[tIdx.y + i][tIdx.x] ← idata[(y + i) * width + x]
5 cs::sync(cta)
6 for i ← 0 to tile_dim by block_rows do:
7 | odata[(y + i) * width + x] = tile[tIdx.x][tIdx.y + i];

```

Algorithm 2: CUDA Transpose Coalesced Coop (CC)

```

1 set shared tile[tile_dim] [tile_dim+1]

```

▷ Same as Algorithm 2, but with padding to avoid bank conflicts

Algorithm 3: CUDA Transpose Coalesced No Conflict (CN-BCC)

```

1 set shared tile[tile_dim][tile_dim+1]
2 let grid ← cg::this_grid()
3 if (x < width and y < width):
4 | tile[tIdx.y][tIdx.x] ← idata[y * width + x]
5 grid.sync()
6 if (x < width and y < width):
7 | odata[y * width + x] ← tile[tIdx.x][tIdx.y]

```

Algorithm 4: CUDA Transpose Inter Block

IV. SYSTEM DESCRIPTION AND EXPERIMENTS

The experiments were conducted on the University of Trento GPU Cluster, equipped with NVIDIA A30 GPUs. Detailed hardware and software specifications are presented in Table 1 and Table 2. Compilation of the code was performed using the `nvcc` compiler with the flags `-O2`, `-code=sm_80`, and `-arch=compute_80`. Hyper-threading is enabled on the cluster.

To mitigate the impact of operating system scheduling and hyper-threading noise, each experiment was repeated 10 times, with each kernel being launched 100 times per experiment. The results were averaged to ensure reliability. This setup is designed to minimize inter-program noise by maintaining consistent priority levels assigned by the operating system during each benchmark. By averaging across multiple runs, the setup also reduces variability caused by potential fluctuations in OS priorities.

The experiments were conducted on matrices with dimensions ranging from $2^2 \times 2^2$ up to $2^{15} \times 2^{15}$.

Component	Version	Docs
GPU	NVidia A30	[3]
Max. theor. bandwidth	933 GB/s	[4]
CPU	Intel XEON Gold 6238R	[5]

Table 1: Hardware Description.

¹ x, y and $width$ are common in all CUDA implementations, for clarity they will be omitted in other pseudocodes.

Component	Version	Docs
OS: Rocky Linux	8.7	[6]
Scheduler: Slurm	20.11.9	[7]
CUDA	12.1.66	[8]
cuBLAS	12.1.0	[2]
C++	14	[9]

Table 2: Software Description².

V. RESULTS

A. Intra-block Synchronization

The results for intra-block synchronization, shown in Figure 1, indicate no significant benefit from using Cooperative Groups for this purpose. A similar trend is observed in Figure 2 and Figure 3, confirming that intra-block synchronization with Cooperative Groups does not enhance performance for matrix transposition. The experimental results are summarized in Table 4.

Interestingly, cuBLAS performs marginally slower than the presented kernels for matrix side sizes in the range $[2^5, 2^{11}]$.

B. Inter-block Synchronization

The results in Figure 4 demonstrate that inter-block synchronization provides no performance benefits for matrix transposition. The data in Table 3 further reinforces this conclusion.

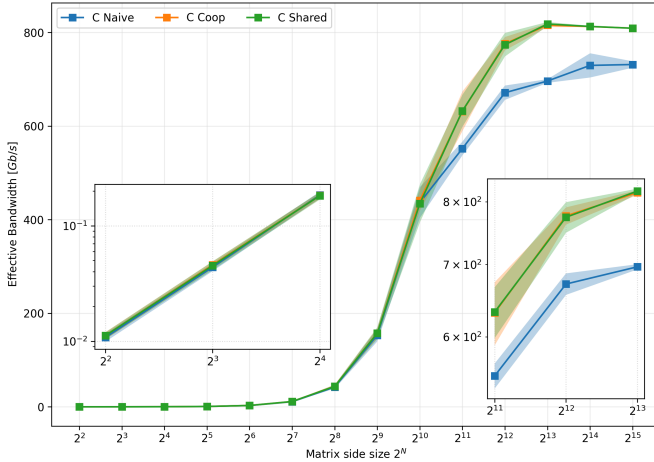


Figure 1: CUDA Matrix Copy Coop. Copy with shared memory and no bank conflicts (C Shared), Copy with shared memory, no bank conflicts, cooperative (C Coop, Algorithm 1). The two kernels using shared memory exhibit similar behavior. The average standard deviation (std) for the C Coop kernel is 7.12 Gbps, for C Shared it is 8.79Gbps, and for C Naive it is 8.36 Gbps.

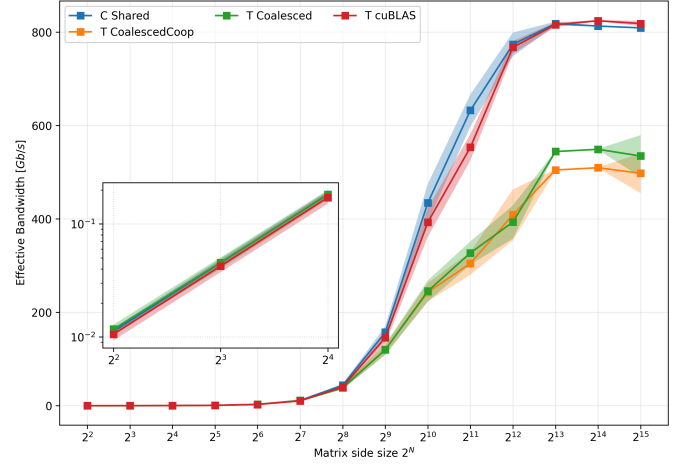


Figure 2: CUDA Matrix Transpose Coalesced Coop. Copy with shared memory (C Shared), Transpose with shared memory (T Coalesced) and Transpose Cooperative with shared memory (T CoalescedCoop, Algorithm 2). Notably, T cublas outperforms all other kernels. Unexpectedly, T CoalescedCoop performs significantly worse than T Coalesced, despite the only difference being the synchronization method.

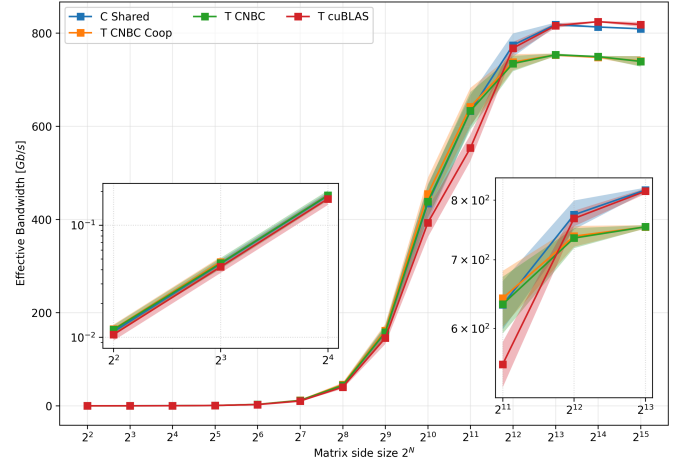


Figure 3: CUDA Matrix Transpose Coalesced No Bank Conflicts Coop. Transpose with Shared Memory, Coalesced No Bank Conflict (T CNBC) and Transpose Cooperative with Shared Memory, Coalesced No Bank Conflict (T CNBC Coop Algorithm 3). For $\mathcal{N} \geq 2^{12}$, T cublas is the best performer. However, within $2^5 \leq \mathcal{N} \leq 2^{11}$, T CNBC Coop outperforms the others.

²CMake was used for local development.

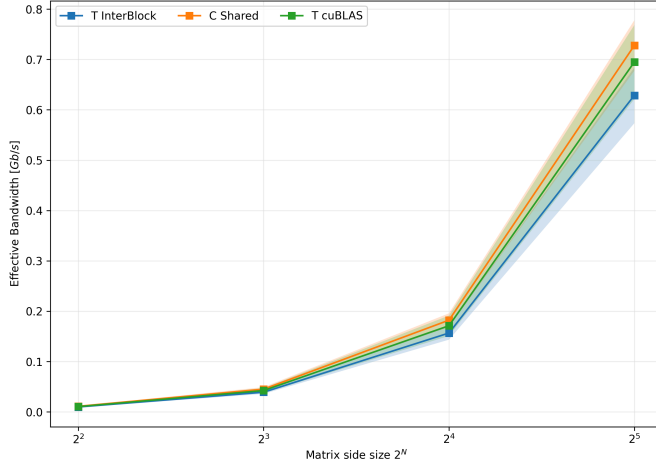


Figure 4: CUDA Transpose Inter Block. T interblock (Algorithm 4). The plot clearly shows that inter-block synchronization is not an effective choice for dense matrix transposition.

Size	InterBlock	cuBLAS
2 ²	0.0098 ± 0.0008	0.0106 ± 0.0012
2 ³	0.0392 ± 0.0033	0.0425 ± 0.005
2 ⁴	0.1568 ± 0.0132	0.1712 ± 0.0202
2 ⁵	0.6288 ± 0.0553	0.6949 ± 0.0734
avg std	0.01815	0.02495

Table 3: Inter-Block Synchronization effective bandwidths. Best results are in **bold**. All measurements are in Gbps.

Size	Coalesced	Coalesced Cooperative	Coalesced No Bank Conflict	Coalesced No Bank Conflict Cooperative	cuBLAS
2 ²	0.01 ± 0	-	0.01 ± 0	0.01 ± 0	0.01 ± 0
2 ³	0.05 ± 0.01	-	0.05 ± 0.01	0.05 ± 0.01	0.04 ± 0.01
2 ⁴	0.18 ± 0.02	-	0.18 ± 0.02	-	0.17 ± 0.02
2 ⁵	0.68 ± 0.07	<u>0.7 ± 0.06</u>	0.74 ± 0.07	0.77 ± 0.08	0.69 ± 0.07
2 ⁶	2.67 ± 0.25	<u>2.73 ± 0.25</u>	2.88 ± 0.26	2.98 ± 0.33	2.62 ± 0.26
2 ⁷	10.52 ± 0.96	<u>10.82 ± 1.16</u>	11.41 ± 1.03	11.78 ± 1.13	10.24 ± 1.01
2 ⁸	38.1 ± 3.16	<u>38.58 ± 3.79</u>	43.95 ± 4.26	45.43 ± 4.27	40.22 ± 3.89
2 ⁹	119.73 ± 11.15	<u>120.31 ± 10.45</u>	157.87 ± 14.52	161.59 ± 14.27	145.37 ± 12.97
2 ¹⁰	<u>245.4 ± 22.72</u>	243.11 ± 19.54	438.17 ± 34.48	454.57 ± 34.61	392.82 ± 31.06
2 ¹¹	<u>327.11 ± 24.79</u>	304.7 ± 24.4	633.16 ± 40.31	642.05 ± 40.29	553.35 ± 28.08
2 ¹²	393.42 ± 35.48	<u>408.88 ± 54.57</u>	734.62 ± 16.46	<u>737.71 ± 17.19</u>	767.68 ± 13.34
2 ¹³	<u>544.33 ± 1.37</u>	504.85 ± 1.44	753.4 ± 2.82	753.16 ± 3.06	816.1 ± 5.01
2 ¹⁴	<u>549.14 ± 0.97</u>	509.65 ± 0.74	749.55 ± 1.49	748.17 ± 2.32	824.54 ± 1.43
2 ¹⁵	<u>534.8 ± 44.6</u>	497.79 ± 42.56	739.21 ± 11.23	<u>740.21 ± 10.56</u>	818.46 ± 5.42
avg std	10.39	14.45	9.06	9.85	7.32

Table 4: Intra-Block Synchronization effective bandwidths. C: coalesced, CC: coalesced cooperative, CNBC and CNBCC: Coalesced No Bank Conflicts (+coop). Best results are in **bold**, the best for each algorithm type are underline. All measurements are in Gbps.

VI. CONCLUSIONS

As anticipated, the application of Cooperative Groups for square dense matrix transposition does not yield significant performance advantages. This holds true for both intra-block synchronization and inter-block synchronization. While cuBLAS does not consistently outperform the custom kernels in terms of raw throughput, it exhibits a consistently lower average standard deviation. This characteristic makes cuBLAS not only competitive but also the “most stable” solution in terms of performance reliability.

These results shed light on the challenges of optimizing matrix transposition. Specifically, the straightforward nature of dense matrix transposition requires less granular control, rendering advanced thread management techniques, such as those offered by Cooperative Groups, less impactful for this task.

Future research could investigate more advanced memory management techniques, such as TMA[10], or optimize memory access patterns to improve kernel efficiency further.

REFERENCES

- [1] NVIDIA, [Online]. Available: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#cooperative-groups>
- [2] NVIDIA, “cuBLAS Library Documentation.” [Online]. Available: <https://docs.nvidia.com/cuda/cublas/index.html>
- [3] NVIDIA, “NVIDIA A30 Specs.” [Online]. Available: <https://www.nvidia.com/en-us/data-center/products/a30-gpu/>
- [4] NVIDIA, [Online]. Available: https://github.com/NVIDIA/cuda-samples/blob/master/Samples/1_Uutilities/bandwidthTest/bandwidthTest.cu#L656
- [5] Intel, “Intel® Xeon® Gold 6238R Processor .” [Online]. Available: <https://www.intel.com/content/www/us/en/products/sku/199345/intel-xeon-gold-6238r-processor-38-5m-cache-2-20-ghz/specifications.html>
- [6] R. Linux, “Rocky Linux 8.7.” [Online]. Available: https://docs.rockylinux.org/release_notes/8_7/
- [7] Slurm, “Slurm Documentation.” [Online]. Available: <https://slurm.schedmd.com/documentation.html>
- [8] NVIDIA, “Cuda Toolkit 12.1.” [Online]. Available: <https://docs.nvidia.com/cuda/archive/12.1.0/cuda-toolkit-release-notes/>
- [9] C. Reference, “C++ 14 Reference.” [Online]. Available: <https://en.cppreference.com/w/cpp/14>
- [10] NVIDIA, “Tensor Memory Access.” [Online]. Available: <https://docs.nvidia.com/cuda/hopper-tuning-guide/index.html#tensor-memory-accelerator>

GPU Computing 2nd deliverable

Ettore Saggiorato

Universita' di Trento

Trento, Italy

ettore.saggiorato@studenti.unitn.it

The code of this project is available on GitHub[1].

I. PROBLEM DESCRIPTION

Develop an algorithm to transpose non-symmetric matrices of size $N \times N$, where N is a power of 2, for both single-threaded CPU and CUDA implementations. Evaluate the CPU implementation's performance by measuring the effective bandwidth under different compiler optimization levels (-O0, -O1, -O2, and -O3), and analyze the cache behavior to assess the implementation's efficiency. For the CUDA implementation, measure the effective bandwidth to evaluate its performance.

II. ALGORITHMS AND KERNELS

The matrix data is defined as a 1D array of size $N \times N$, where the element at position i, j is stored at the index $i \times N + j$. The matrix is stored in row-major order¹. The CUDA implementation uses a 2D grid of blocks and threads to transpose the matrix.

```
1 let result ← Matrix shape.y × shape.x
2 for i ← 0 to shape.x do:
3   for j ← 0 to shape.y do:
4     result[j][i] ← result[i][j]
5 return result
```

Algorithm 1: Simple Transposition

```
1 let result ← Matrix shape.y × shape.x
2 let N ← shape.x
3 let M ← shape.y
4 for (ii ← 0) to N by block do:
5   for (jj ← 0) to M by block do:
6     for (i ← ii to min(ii+block, N)) do:
7       for (j ← jj to min(jj+block, M)) do:
8         result[j][i] ← data[i][j]
9 return result
```

Algorithm 2: Block Matrix Transposition

```
1 let x ← blockIdx.x × tile_dim + threadIdx.x
2 let y ← blockIdx.y × tile_dim + threadIdx.y
3 let width ← gridDim.x × tile_dim2
4 for i ← 0 to tile_dim by block_rows do:
5   odata[(y + i) × width + x] ← idata[(y + i) × width + x]
```

Algorithm 3: CUDA Copy

```
1 for i ← 0 to tile_dim by block_rows do:
2   odata[width × x + (y + i)] = idata[(y + i) × width + x];
```

Algorithm 4: CUDA Transpose Naive

```
1 set shared tile[tile_dim][tile_dim]
2 for i ← 0 to tile_dim by block_rows do:
3   tile[tIdx.y + i][tIdx.x] ← idata[(y + i) × width + x]
4 syncthreads
5 for i ← 0 to tile_dim by block_rows do:
6   odata[(y + i) × width + x] = tile[tIdx.x][tIdx.y + i];
```

Algorithm 5: CUDA Transpose Shared

```
1 set shared tile[tile_dim] // Same as CUDA Transpose
                             Shared, but with padding to
                             [tile_dim+1] avoid bank conflicts
```

Algorithm 6: CUDA Transpose Shared No Conflicts

A. CPU Transposition

Algorithm 1 is straightforward and easy to implement. However, due to its implementation it is not cache-friendly, as it does not exploit the cache hierarchy.

Algorithm 2 divides the matrix into blocks of size $\text{block} \times \text{block}$ and transposes each block. This approach improves the cache behavior by reducing the number of cache misses.

B. CUDA Transposition

The performance of various transposition algorithms is evaluated using the Algorithm 3 kernel as a baseline. This straightforward implementation only utilizes global memory and exploits memory coalescing, meaning all threads read and

¹matrix[i][j] notation is used instead of matrix[i × shape.y + j] due to space constraints and readability concerns.

²x, y and width are common in all CUDA implementations, they will be omitted in other pseudocodes.

write to contiguous memory locations. However, it does not leverage shared memory.

The Algorithm 4 is a naive transposition implementation that also uses only global memory. While it exploits memory coalescing during read operations, it does not do so during writes. The strided memory access pattern during writes results in suboptimal performance and poor effective bandwidth, as noted in [2].

The Algorithm 5 improves upon the naive implementation by using shared memory. It extracts a 2D tile of a multidimensional array from global memory in a coalesced fashion into shared memory, and then have contiguous threads stride through the shared memory tile, without losing any performance. After memory loading, the threads need to synchronize as they will read strided memory locations from shared memory. This is done so that all of them can write to global memory in a coalesced fashion. This approach removes any striding while accessing global memory and improves the effective bandwidth. However, it is still not optimal due to bank conflicts. Bank conflicts occur when multiple threads in a warp access different addresses within the same memory bank, causing serialized access and reducing throughput. The presence of bank conflicts can be detected using the CUDA profiler or through exclusion, as detailed in [3], but this analysis is omitted here for brevity.

Finally, Algorithm 6 includes padding in shared memory ensures that memory accesses within a warp are spread across different memory banks, preventing bank conflicts.

III. PERFORMANCE METRICS

On the cpu runs time is registered using the `std::chrono` library, while on CUDA with `cudaEvent`. The effective bandwidth is calculated as the ratio between the number of bytes read and written and the time taken to perform the operation using the formula:

$$eb = \text{effective bandwidth} = \frac{Br + Bw}{t_{\text{routine}}} \quad (1)$$

That in the case of matrix transpose is equal to:

$$eb = \frac{\text{size}_A \times \text{size}_B \times \text{dtype}_{\text{bytes}}}{\text{time} \times 10^9} \quad (2)$$

GPU's theoretical bandwidth is calculated as:

$$\frac{\text{mem_clock_rate} \times 10^3 \times \left(\frac{\text{memory_bus_width}}{8} \right) \times 2}{10^9} \quad (3)$$

and is specified in the Table 3.

IV. EXPERIMENTAL SETUP

A. Hardware

The experiments were performed on a system equipped with a robust CPU cooler, ensuring that thermal throttling was

effectively prevented. Table 1, Table 2, Table 3 detail the CPU, memory and graphics card specifications respectively.

Model	AMD Ryzen 5 5600X
Architecture	x86
Clock Speeds	3.7 GHz base, 4.6 GHz boost
Cache Levels	L1 384 KB, L2 3 MB, L3 32 MB
Cores, Threads	6, 12

Table 1: CPU Specs.

Type	DDR4
Size	16 GB
Speed	3200 MHz
Memory Channels	Dual Channel
DOCP/AMP/XMP	DOCP 3200 MHz

Table 2: RAM Specs.

Device 0	NVIDIA GeForce GTX 1060 6GB
CUDA Driver/Runtime	12.4 / 12.5
CUDA Capability	6.1
Total global memory	6043 MBytes
Multiprocessors, Cores/MP	10, 128, 1280 CUDA Cores
Memory Clock rate	4004 Mhz
Memory Bus Width	192-bit
Total shared memory per block	49152 bytes
Warp size	32
Max # of threads per block	1024
Max size of a thread block (x,y,z)	(1024, 1024, 64)
Theoretical memory bandwidth	192.2 GBps

Table 3: GPU Specs.³

V. CPU RUNS

To improve the accuracy of the measurements and reduce the OS noise, each experiment was repeated multiple times, in function of the specific fun, to have ~1s run time, and the average value was calculated. The matrix size was varied from 2^1 to 2^{10} , and the block size was set to 16. All experiments were done with float data type. Results are presented in the next section.

³Informations from NVidia's deviceQuery tool [4].

A. Performance Trends

Taking a look at Figure 1 we can notice that:

- No optimization exhibits the worst performance, which is expected as it applies no optimization techniques.
- O1 optimization shows a similar curve to no optimization but with slightly better performance. Both methods perform comparably.
- O2 and O3 optimizations demonstrate a more significant improvement globally.
- The simple method outperforms the block method for small matrices ($\leq 2^6$), while the block method performs better for larger matrices. This difference persists despite similar cache hit and miss rates for both methods. This may be due to overhead of block management.

B. Cache Comparison

Comparing cache characteristics between O3 simple and block optimizations from Figure 3, Figure 4, and Figure 5:

- Both methods exhibit similar instruction, data, and unified cache behavior.
- However, there's a noticeable difference in cache misses, particularly in the D1 miss rate, with O3 simple at 3.3% and O3 block at 1.3% when $N = 2^8$.

1) **Decrease in Cache References:** From Figure 6 we can observe that as optimization levels increase, cache references decrease. This reduction is attributed to more efficient code resulting from higher optimization levels, leading to fewer memory accesses. This decrease is evident in D references. Note that the D1 miss rate increases as the matrix size increases and as the optimization level increases. This increase is primarily in the write cache, suggesting that prefetching may be beneficial.

2) **Similar Cache Miss:** For matrices with size $\leq 10^6$, all optimization levels exhibit similar cache miss behavior. However, differences become more pronounced for matrices with size $> 10^7$.

3) **Performance Variation between Simple and Block Methods:** The performance difference between the simple and block methods stems from various factors:

- **Cache Line Utilization:** The block method operates on smaller blocks that fit within a single cache line, leading to better cache line utilization. In contrast, the simple method may access larger contiguous regions, resulting in more cache misses.
- **Spatial Locality:** The block method exhibits better spatial locality as it accesses data in a more localized manner within each block. Conversely, the simple method may display poorer spatial locality, especially for larger matrices.

Overall, the difference in miss rates between the simple and block methods for larger matrices is a combination of cache line utilization, spatial locality, and loop optimizations.

VI. CUDA

The GPU runs were repeated 100 times to minimize OS and system noise. The grid size was calculated dynamically based on the matrix size, while the block size was kept static. Multiple block dimensions were tested: $[16 \times 4]$, $[16 \times 8]$, $[16 \times 16]$, $[32 \times 4]$, $[32 \times 8]$, $[32 \times 16]$, to assess kernel sensitivity to block size. Figure 7 illustrates the effective bandwidth performance of the different kernels, demonstrating their sensitivity to block size. Additional plots are available in the repository. The naive implementation, due to its lack of shared memory usage, is less affected by block size variations.

The speed-up relative to Algorithm 2 is approximately an order of magnitude, and about 4 times faster compared to Algorithm 4. This highlights the significant performance gains achievable with GPUs for parallelizable matrix operations.

When comparing Algorithm 5 and Algorithm 6, it is evident that bank conflicts significantly impacted the performance of the former kernel.

VII. FUTURE WORK

When comparing multiple matrix sizes, the shared implementation without bank conflicts proves to be faster than a simple copy kernel without shared memory. Introducing a comparison with a shared-memory copy kernel would enhance this analysis.

Further investigation into the diagonal block reordering kernel proposed by NVidia[5] could provide valuable insights. A comparison with existing implementations and an exploration of optimal usage scenarios would be beneficial.

All implementations discussed here spawn $\frac{\text{shape.x}}{\text{TILE_DIM}} \times \frac{\text{shape.y}}{\text{TILE_DIM}}$ blocks, each with $\text{TILE_DIM} \times \text{BLOCK_ROWS}$ threads. However, if the number of threads exceeds the maximum concurrently running threads of the GPU, performance may suffer due to queuing. Therefore, exploring a kernel that maximizes the utilization of concurrent threads, evenly distributes tasks among threads, maintains coalesced memory access, and avoids shared memory bank conflicts could be interesting.

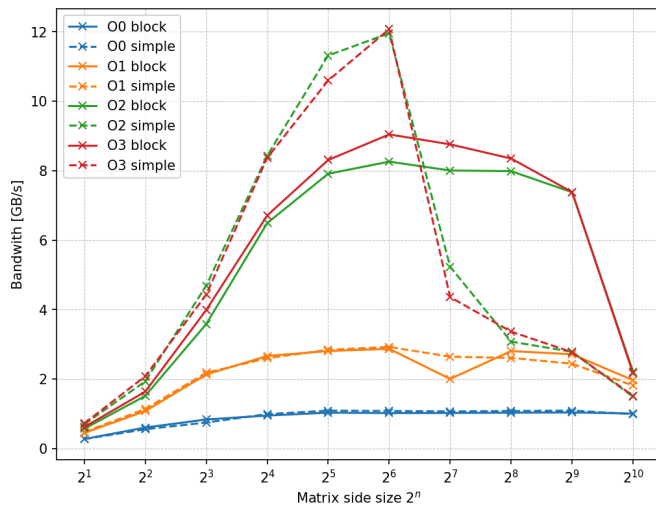


Figure 1: CPU - Effective bandwidth (float)

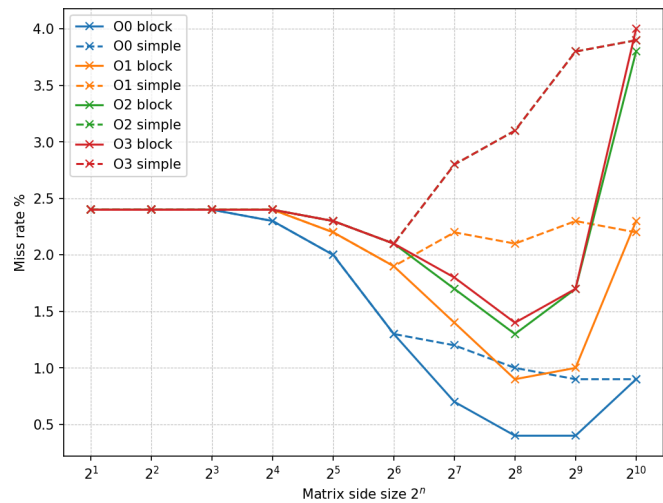


Figure 4: CPU - D1 miss rate

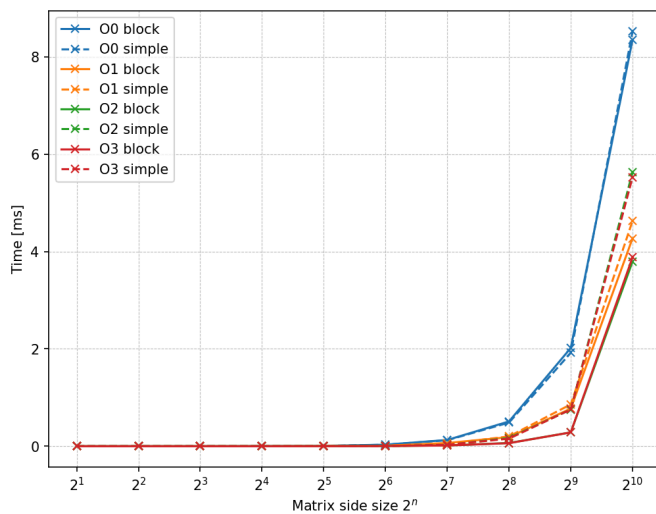


Figure 2: CPU - Execution time (float)

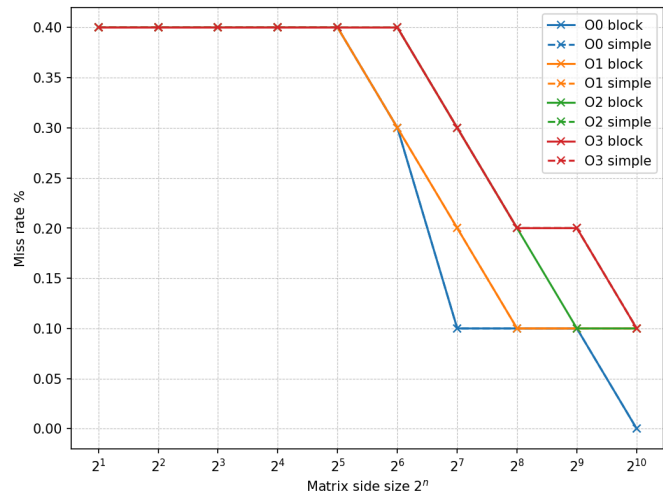


Figure 5: CPU - LL miss rate

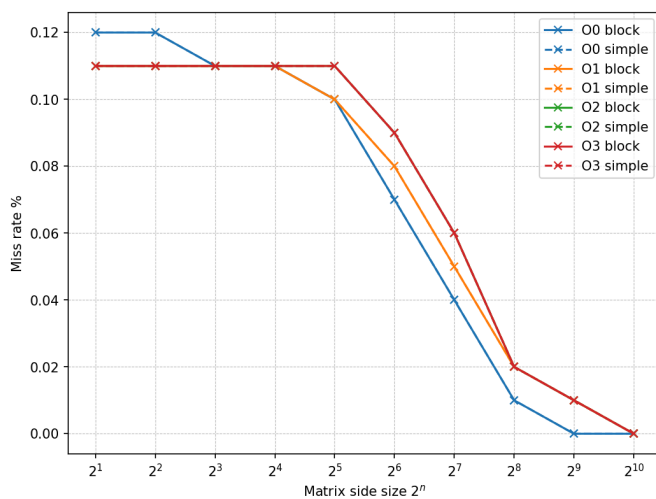


Figure 3: CPU - L1 miss rate

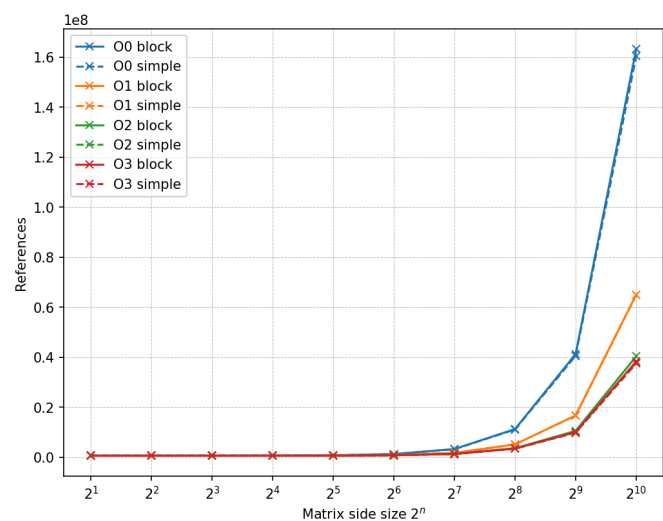


Figure 6: CPU - D references

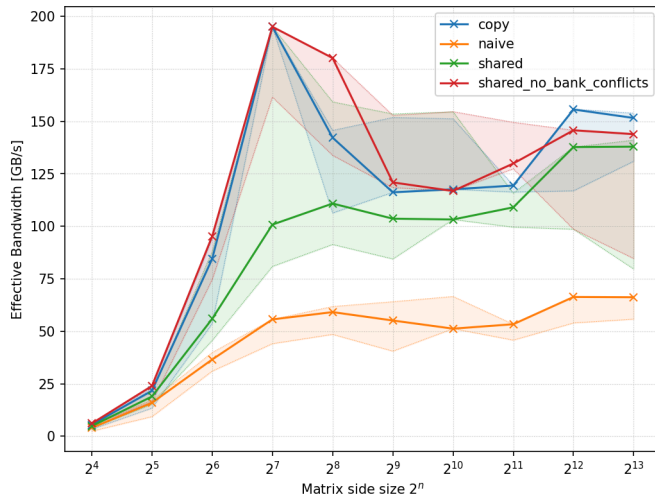


Figure 7: CUDA - Effective bandwidth - Performance of matrix transpose implementations (copy, naive, shared, shared_no_bank_conflicts). Shaded areas show the min-max range from multiple grid size dimensions, with size 32x8 highlighted.

REFERENCES

- [1] S. - Ettore Saggiorato, "GPU Computing." [Online]. Available: <https://github.com/sa1g/gpu-computing>
- [2] M. Harris, "How to Access Global Memory Efficiently in CUDA C/C++ Kernels." [Online]. Available: <https://developer.nvidia.com/blog/how-access-global-memory-efficiently-cuda-c-kernels/>
- [3] M. Harris, "An Efficient Matrix transpose in CUDA C/C++." [Online]. Available: <https://developer.nvidia.com/blog/efficient-matrix-transpose-cuda-cc/>
- [4] N. Corporation, "CUDA Samples - Device Query." [Online]. Available: https://github.com/NVIDIA/cuda-samples/tree/master/Samples/1_Uilities/deviceQuery
- [5] G. Ruetsch and P. Micikevicius, "Optimizing Matrix Transpose in CUDA." [Online]. Available: <https://www.cs.colostate.edu/~cs675/MatrixTranspose.pdf>

GitHub repository

1 Problem Description

Develop an algorithm to transpose non-symmetric matrices of size $N \times N$, where N is a power of 2. Assess the algorithm's performance by measuring effective bandwidth under different compiler optimization options (-O0, -O1, -O2, and -O3). Analyze cache behavior to evaluate implementation efficiency.

1.1 Matrix definition

The matrix data is defined as a 1D array of size $N \times N$, where the element at position i, j is stored at the index $i \times N + j$. The matrix is stored in row-major order.

1.2 Matrix transposition¹

Algorithm 1 Simple matrix transposition

```
function T0
  result ← Matrix shape.y × shape.x
  for i ← 0 to shape.x do
    for j ← 0 to shape.y do
      result[j][i] ← data[i][j]
    end for
  end for
  return result
end function
```

Algorithm 2 Block Matrix Transpose

```
function T2(block = 16)
  result ← Matrix shape.y × shape.x
  N ← shape.x
  M ← shape.y
  for ii ← 0 to N by block do
    for jj ← 0 to M by block do
      for i ← ii to min(ii + block, N) do
        for j ← jj to min(jj + block, M) do
          result[j][i] ← data[i][j]
        end for
      end for
    end for
  end for
  return result
end function
```

The simple transposition algorithm (Algorithm 1) is straightforward and easy to implement. However, due to its implementation it

¹ $matrix[i][j]$ notation is used instead of $matrix[i \times shape.y + j]$ due to space constraints and readability concerns.

is not cache-friendly, as it does not exploit the cache hierarchy. The block matrix transposition algorithm (Algorithm 2) divides the matrix into blocks of size $block \times block$ and transposes each block. This approach improves the cache behavior by reducing the number of cache misses.

1.3 Optimization options

O0, O1, O2, O3 description

- -O0 No optimization.
- -O1 Optimize.
- -O2 Optimize even more.
- -O3 Optimize yet more.

1.4 Performance metrics

Time is registered using the `std::chrono` library. The effective bandwidth is calculated as the ratio between the number of bytes read and written and the time taken to perform the operation using the formula:

$$eb = \text{Effective bandwidth} = \frac{Br + Bw}{t_{\text{routine}}} \quad (1)$$

That in the case of matrix transpose is equal to:

$$eb = \frac{\text{size}_A \times \text{size}_B \times \text{dtype_bytes}}{\text{time} \times 10^9} \quad (2)$$

2 Experimental setup

2.1 Hardware

The experiments were performed on a system equipped with a robust CPU cooler, ensuring that thermal throttling was effectively prevented. Table 1 and Table 2 detail the CPU and memory specifications, respectively.

Model	AMD Ryzen 5 5600X
Architecture	x86
Clock Speeds	3.7 GHz base, 4.6 GHz boost
Cache Levels	L1 384 KB, L2 3 MB, L3 32 MB
Cores, Threads	6, 12

Table 1: Processor specs.

Type	DDR4
Size	16 GB
Speed	3200 MHz
Memory Channels	Dual Channel
DOCP/AMP/XMP	DOCP 3200 MHz

Table 2: Memory specs.

2.2 Runs

To improve the accuracy of the measurements and reduce the OS noise, each experiment was repeated multiple times, in function of the specific fun, to have 1s run time, and the average value was calculated. The matrix size was varied from 2^1 to 2^{10} , and the block size was set to 16. All experiments were done with float data type. Results are presented in the next section.

3 Results

Performance Trends

Taking a look at Figure 1 we can notice that:

- No optimization exhibits the worst performance, which is expected as it applies no optimization techniques.
- O1 optimization shows a similar curve to no optimization but with slightly better performance. Both methods perform comparably.
- O2 and O3 optimizations demonstrate a more significant improvement globally.
- The simple method outperforms the block method for small matrices ($\leq 2^6$), while the block method performs better for larger matrices. This difference persists despite similar cache hit and miss rates for both methods. This may be due to overhead of block management.

Cache Comparison

Comparing cache characteristics between O3 simple and block optimizations from Figure 3, 4, and 5:

- Both methods exhibit similar instruction, data, and unified cache behavior.
- However, there's a noticeable difference in cache misses, particularly in the D1 miss rate, with O3 simple at 3.3% and O3 block at 1.3% when $N = 2^8$.

Decrease in Cache References

From Figure 6 we can observe that as optimization levels increase, cache references decrease. This reduction is attributed to more efficient code resulting from higher optimization levels, leading to fewer memory accesses. This decrease is evident in D references. Note that the D1 miss rate increases as the matrix size increases and as the optimization level increases. This increase is primarily in the write cache, suggesting that prefetching may be beneficial.

Similar Cache Miss

For matrices with size $\leq 10^6$, all optimization levels exhibit similar cache miss behavior. However, differences become more pronounced for matrices with size $> 10^7$.

Performance Variation between Simple and Block Methods

The performance difference between the simple and block methods stems from various factors:

- **Cache Line Utilization:** The block method operates on smaller blocks that fit within a single cache line, leading to better cache line utilization. In contrast, the simple method may access larger contiguous regions, resulting in more cache misses.
- **Spatial Locality:** The block method exhibits better spatial locality as it accesses data in a more localized manner within each block. Conversely, the simple method may display poorer spatial locality, especially for larger matrices.

Overall, the difference in miss rates between the simple and block methods for larger matrices is a combination of cache line utilization, spatial locality, and loop optimizations.

4 Future work

The most straightforward method to speed up the transposition process is to parallelize the algorithm. This can be achieved using a block method, where each block is transposed in parallel. This approach can be implemented using OpenMP or CUDA for CPUs and GPUs, respectively. Since the transpose operation operates on a distinct matrix, the algorithm can directly manipulate memory without necessitating the use of semaphores or mutexes.

GPU Computing - Assignment 1

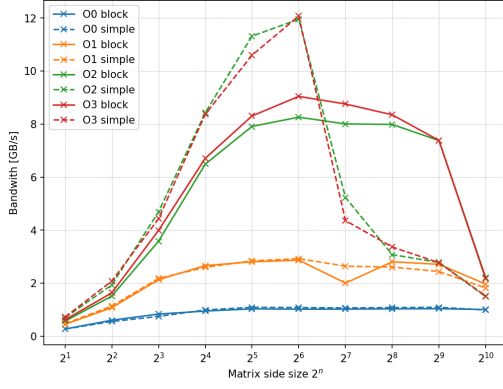


Figure 1: Effective bandwidth (float)

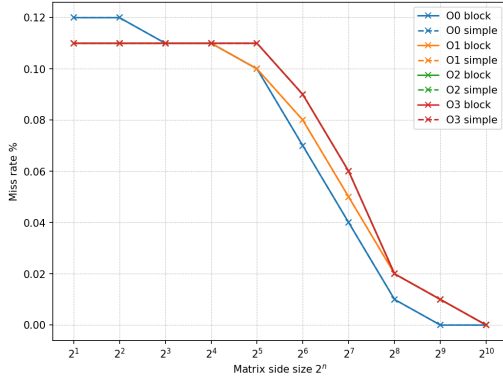


Figure 3: L1 miss rate

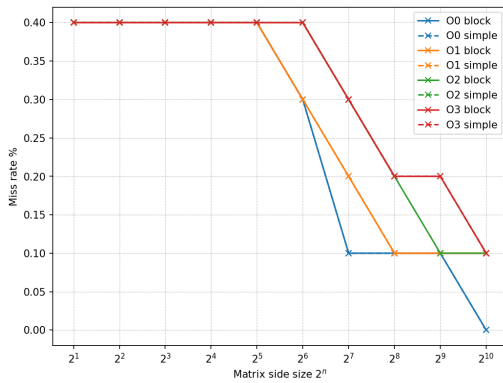


Figure 5: LL miss rate

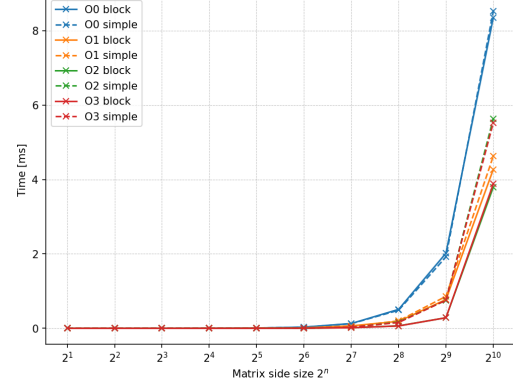


Figure 2: Execution time (float)

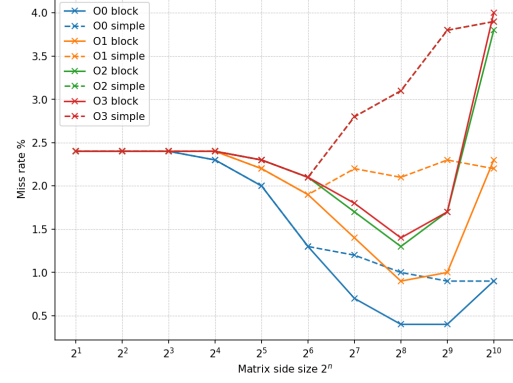


Figure 4: D1 miss rate

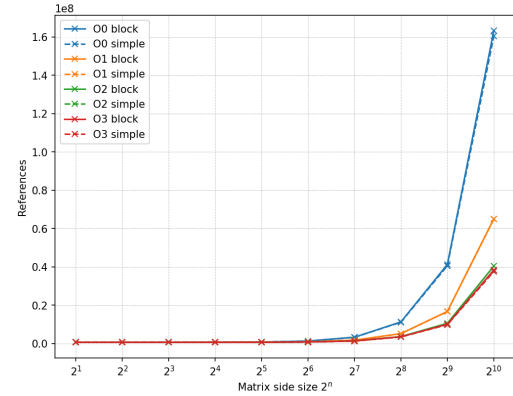


Figure 6: D references